



南開大學
Nankai University

数据结构实验报告五

散列表

学号：2412266

姓名：杨滢

专业：信息安全

学院：密码与网络空间安全学院

目录

1 题目一	2
1.1 题目表述	2
1.2 代码 1 的测试用例	2
1.3 思路	2
1.3.1 插入操作	3
1.3.2 查找操作	3
1.3.3 删除操作	3
1.3.4 重组机制	3
1.3.5 命令解析	4
1.4 neverUsed 机制验证	4
1.5 代码	4
1.6 测试用例截图	10
1.7 复杂度分析	10
2 题目二	11
2.1 题目表述	11
2.2 代码 2 的测试用例	11
2.3 思路	12
2.3.1 构造函数	12
2.3.2 插入操作	12
2.3.3 查找操作	12
2.3.4 删除操作	13
2.3.5 尾哨兵验证	13
2.4 测试用例截图	13
2.5 代码	13
2.6 时间性能比较	20
2.6.1 hashChainsWithTail:	20
2.6.2 hashChains:	20

1 题目一

1.1 题目表述

开发一个基于线性探查的散列表类, 要求用 neverUsed 思想进行删除操作。为每个方法编写 C++ 代码。其中有一个方法, 它在 60% 的空桶的 neverUsed 域的值 false 时, 重新组织散列表。重新组织散列表的过程要在必要时移动记录, 重新组织之后, 每个空桶的 neverUsed 域的值 true。测试代码的正确性。

1.2 代码 1 的测试用例

容量与哈希: $\text{cap} = 7$, $h(k) = k \% 7$

1. insert(1,"a") {inserted, index:1}
2. insert(8,"b") {inserted, index:2}
3. insert(15,"c") {inserted, index:3}
4. find(22) {not_found, index:4}
5. erase(8) {removed, index:2}
6. find(15) {found, index:3, value:"c"}
7. insert(22,"d") {inserted, index:2}
8. insert(0,"t0") {inserted, index:0}
9. insert(2,"t2") {inserted, index:4}
10. insert(4,"t4") {inserted, index:5}
11. erase(0) {removed, index:0}
12. erase(2) {removed, index:4}
13. erase(4) {removed, index:5}
14. insert(5,"e") 触发重组织后插入 $\rightarrow$ {inserted, index:5}
15. find(1) {found, index:1, value:"a"}
16. find(22) {found, index:2, value:"d"}
17. find(0) {not_found, index:0}

1.3 思路

基于动态数组实现线性探查散列表。

1.3.1 插入操作

1. 插入前检查是否需要重组织 (60% 条件)
2. 计算初始桶号: $b = \text{key} \% 7$
3. 从 `table[b]` 开始线性探查:
 - 如果遇到空桶 ($\text{key} == -1$) $\rightarrow$ 插入新元素, 设置 `neverUsed = false`
 - 如果遇到相同 `key` $\rightarrow$ 更新 `value`, 返回 `exists`
 - 如果表满 $\rightarrow$ 返回插入失败
4. 返回插入结果 `inserted/exists, index`

1.3.2 查找操作

1. 计算初始桶号 $b = \text{key} \% 7$
2. 从 `table[b]` 开始线性探查:
 - 如果遇到 `neverUsed` 为 `true` 的空桶 $\rightarrow$ 停止搜索, 返回 `not_found`
 - 如果找到匹配 `key` $\rightarrow$ 返回 `found` 和对应值
 - 如果探查完整个表 $\rightarrow$ 返回 `not_found`
3. 返回查找结果 `found/not_found, index, value`

1.3.3 删除操作

1. 计算初始桶号 $b = \text{key} \% 7$
2. 从 `table[b]` 开始线性探查:
 - 如果遇到 `neverUsed` 为 `true` 的空桶 $\rightarrow$ 停止搜索, 返回 `not_found`
 - 如果找到匹配 `key` $\rightarrow$ 执行删除
3. 删除操作:
 - 设置 `key = -1, value = ""` (标记为空桶)
 - 保持 `neverUsed = false` (表示曾经被使用)
 - 元素计数 `size` 减 1
4. 返回删除结果 `removed/not_found, index`

1.3.4 重组织机制

1. 触发条件:
 - 统计所有空桶数量 `emptyCount`
 - 统计 `neverUsed` 为 `false` 的空桶数量 `falseNeverUsedCount`
 - 当 $\frac{\text{falseNeverUsedCount}}{\text{emptyCount}} \geq 0.6$ 时触发重组织

2. 重组过程:

- 保存当前所有有效元素到临时数组
- 重新初始化散列表（所有桶的 neverUsed 设为 true）
- 将有效元素重新插入到新表中
- 保持原有的哈希和线性探查规则

3. 重组效果：所有空桶的 neverUsed 域恢复为 true

1.3.5 命令解析

1. 输入格式:

- insert(key,"value")
- find(key)
- erase(key)

2. 解析过程:

- 去除所有空格字符
- 识别命令类型（insert/find/erase）
- 提取 key 和 value 参数
- 调用对应的散列表方法

3. 输出格式：统一返回操作结果, index, 附加信息

1.4 neverUsed 机制验证

1. 新建的表：所有桶的 neverUsed 初始化为 true
2. 插入操作：被使用的桶设置 neverUsed = false
3. 删除操作：保持 neverUsed = false（标记曾经使用）
4. 重组后：所有空桶的 neverUsed 恢复为 true
5. 查找优化：遇到 neverUsed 为 true 的空桶即可停止搜索

1.5 代码

```
1 #include <iostream>
2 #include <vector>
3 #include <string>
4 #include <sstream>
5 using namespace std;
6 class LinearProbingHashTable {
7 private:
```

```
8 struct Entry {
9     int key;
10    string value;
11    bool neverUsed;
12
13    Entry() : key(-1), value(""), neverUsed(true) {}
14 };
15 vector<Entry> table;
16 const int capacity=7;
17 int size;
18 int hash(int key) const {
19     return key % capacity;
20 }
21 bool needReorganize() const {
22     int emptyCount = 0;
23     int falseNeverUsedCount = 0;
24     for (const auto& entry : table) {
25         if (entry.key == -1) {
26             emptyCount++;
27             if (!entry.neverUsed) {
28                 falseNeverUsedCount++;
29             }
30         }
31     }
32     if (emptyCount == 0) return false;
33     return (static_cast<double>(falseNeverUsedCount) / emptyCount) >= 0.6;
34 }
35 void reorganize() {
36     vector<Entry> oldTable = table;
37     table = vector<Entry>(capacity);
38     size = 0;
39
40     for (const auto& entry : oldTable) {
41         if (entry.key != -1) {
42             insertInternal(entry.key, entry.value);
43         }
44     }
45 }
46 bool insertInternal(int key, const string& value) {
47     int index = hash(key);
48     int startIndex = index;
49     do {
```

```
50         if (table[index].key == -1) {
51             table[index].key = key;
52             table[index].value = value;
53             table[index].neverUsed = false;
54             size++;
55             return true;
56         }
57         index = (index + 1) % capacity;
58     } while (index != startIndex);
59     return false;
60 }
61
62 public:
63     LinearProbingHashTable(int initialCapacity = 7) : capacity(initialCapacity),
64         size(0) {
65         table.resize(capacity);
66     }
67     struct InsertResult {
68         bool inserted;
69         int index;
70     };
71     InsertResult insert(int key, const string& value) {
72         if (needReorganize()) {
73             reorganize();
74         }
75         int index = hash(key);
76         int startIndex = index;
77         do {
78             if (table[index].key == -1) {
79                 table[index].key = key;
80                 table[index].value = value;
81                 table[index].neverUsed = false;
82                 size++;
83                 return { true, index };
84             }
85             else if (table[index].key == key) {
86                 table[index].value = value;
87                 return { false, index };
88             }
89             index = (index + 1) % capacity;
90         } while (index != startIndex);
```

```
91     return { false, -1 };
92 }
93 struct FindResult {
94     bool found;
95     int index;
96     string value;
97 };
98 FindResult find(int key) {
99     int index = hash(key);
100    int startIndex = index;
101    do {
102        if (table[index].neverUsed) {
103            return { false, index, "" };
104        }
105        if (table[index].key == key) {
106            return { true, index, table[index].value };
107        }
108        index = (index + 1) % capacity;
109    } while (index != startIndex && !table[index].neverUsed);
110    return { false, index, "" };
111 }
112 bool erase(int key) {
113     int index = hash(key);
114     int startIndex = index;
115     do {
116         if (table[index].neverUsed) {
117             return false;
118         }
119         if (table[index].key == key) {
120             table[index].key = -1;
121             table[index].value = "";
122             size--;
123             return true;
124         }
125         index = (index + 1) % capacity;
126     } while (index != startIndex && !table[index].neverUsed);
127     return false;
128 }
129 };
130
131 void parseCommand(const string& cmd, LinearProbingHashTable& ht) {
132     string cleanCmd;
```



```
133     for (char c : cmd) {
134         if (c != ' ') cleanCmd += c;
135     }
136     if (cleanCmd.empty()) return;
137     if (cleanCmd.find("insert(") == 0) {
138         size_t start = cleanCmd.find('(') + 1;
139         size_t comma = cleanCmd.find(',');
140         size_t end = cleanCmd.find(')');
141         if (comma == string::npos || end == string::npos) {
142             cout << "格式错误" << endl;
143             return;
144         }
145         string keyStr = cleanCmd.substr(start, comma - start);
146         string valueStr = cleanCmd.substr(comma + 1, end - comma - 1);
147         if (valueStr.front() == '"' && valueStr.back() == '"') {
148             valueStr = valueStr.substr(1, valueStr.length() - 2);
149         }
150         int key = stoi(keyStr);
151         auto result = ht.insert(key, valueStr);
152         if (result.inserted) {
153             cout << "{inserted, index:" << result.index << "}" << endl;
154         }
155         else {
156             cout << "{exists, index:" << result.index << "}" << endl;
157         }
158     }
159     else if (cleanCmd.find("find(") == 0) {
160         size_t start = cleanCmd.find('(') + 1;
161         size_t end = cleanCmd.find(')');
162         string keyStr = cleanCmd.substr(start, end - start);
163         int key = stoi(keyStr);
164         auto result = ht.find(key);
165         if (result.found) {
166             cout << "{found, index:" << result.index << ", value:\"\" << result.value
167                 << "\"}" << endl;
168         }
169         else {
170             cout << "{not_found, index:" << result.index << "}" << endl;
171         }
172     }
173     else if (cleanCmd.find("erase(") == 0) {
174         size_t start = cleanCmd.find('(') + 1;
```

```
174     size_t end = cleanCmd.find(' ');
175     if (end == string::npos) {
176         cout << "格式错误" << endl;
177         return;
178     }
179     string keyStr = cleanCmd.substr(start, end - start);
180     int key = stoi(keyStr);
181     auto findResult = ht.find(key);
182     if (ht.erase(key)) {
183         cout << "{removed, index:" << findResult.index << "}" << endl;
184     }
185     else {
186         cout << "{not_found, index:" << findResult.index << "}" << endl;
187     }
188 }
189 else {
190     cout << "未知命令" << endl;
191 }
192 }
193 int main() {
194     LinearProbingHashTable ht(7);
195     string command;
196     while (true) {
197         getline(cin, command);
198         if (command == "exit" || command == "quit") {
199             break;
200         }
201         parseCommand(command, ht);
202     }
203     system("pause");
204     return 0;
205 }
```

1.6 测试用例截图

```
insert(1,"a")
{inserted, index:1}
insert(8,"b")
{inserted, index:2}
insert(15,"c")
{inserted, index:3}
find(22)
{not_found, index:4}
erase(8)
{removed, index:2}
find(15)
{found, index:3, value:"c"}
insert(22,"d")
{inserted, index:2}
insert(0,"t0")
{inserted, index:0}
insert(2,"t2")
{inserted, index:4}
insert(4,"t4")
{inserted, index:5}
erase(0)
{removed, index:0}
erase(2)
{removed, index:4}
erase(4)
{removed, index:5}
insert(5,"e")
{inserted, index:5}
find(1)
{found, index:1, value:"a"}
find(22)
{found, index:2, value:"d"}
find(0)
{not_found, index:0}
```

1.7 复杂度分析

表 1: 各操作时间复杂度分析

操作	最好情况	平均情况
重组织操作	$O(1)$	$O(1)$
插入操作	$O(1)$	$O(1)$
查找操作	$O(1)$	$O(1)$
删除操作	$O(1)$	$O(1)$
命令解析	$O(n)$	$O(n)$

n 为输入命令长度。

表 2: 空间复杂度详细分析

组件	空间复杂度	具体大小
散列表数组	$O(capacity)$	7 个 Entry
每个 Entry 结构	$O(1)$	int + string + bool
容量变量	$O(1)$	1 个 int
大小变量	$O(1)$	1 个 int
重组织临时数组	$O(capacity)$	7 个 Entry
总空间复杂度	$O(1)$	固定大小

2 题目二

2.1 题目表述

设计一个类 `hashChainsWithTail`, 其中每个散列链表都是一个有尾节点的有序链表, 而且所有链表在物理上都共享一个尾节点。不使用任何链表类的方法实现插入和删除。和类 `hashChains`(见程序 10-20) 比较时间性能。

程序 10-29:

```
1 template<class K, class E>
2 pair<const K, E>* find(const K& theKey) const{
3     return table[hash(theKey) & divisor].find(theKey);
4 }
5 void insert(const pair<const K, E>& thePair) {
6     int homeBucket = (int)hash(thePair.first) & divisor;
7     int homeSize = table[homeBucket].size();
8     table[homeBucket].insert(thePair);
9     if (table[homeBucket].size() > homeSize)
10         dSize++;
11 }
12 void erase(const K& theKey){
13     table[hash(theKey) & divisor].erase(theKey);
14 }
```

2.2 代码 2 的测试用例

容量与哈希: $cap=5$, $h(k)=k\%5$

1. `insert(6,"a")` {inserted, bucket:1, pos:0}
2. `insert(1,"b")` {inserted, bucket:1, pos:0}
3. `insert(11,"c")` {inserted, bucket:1, pos:2}
4. `find(6)` {found, bucket:1, pos:1}
5. `erase(1)` {removed, bucket:1}
6. `erase(7)` {not_found, bucket:2}
7. `insert(16,"d")` {inserted, bucket:1, pos:2}
8. `insert(6,"A'")` {exists, bucket:1, pos:0}
9. `find(99)` {not_found, bucket:4}
10. 遍历检查尾哨兵对每个桶: `tail.next==tail` 且 `&tail` 同 `addr(T)` (同一地址)

2.3 思路

2.3.1 构造函数

1. 创建全局尾哨兵 T ，设置 $T.next = T$ （自指向）
2. 分配 5 个桶的指针数组
3. 将所有桶初始化为指向 T （表示空链表）

2.3.2 插入操作

1. 计算桶号 $b = key$
2. 从 $table[b]$ 开始遍历链表
3. 遍历过程中维护两个指针：
 - $current$: 当前检查的节点
 - $prev$: $current$ 的前驱节点
4. 遍历时的三种判断：
 - 如果 $current.key == \text{新 key}$ → 返回 $exists$ （不插入）
 - 如果 $current.key > \text{新 key}$ → 找到插入位置，跳出循环
 - 如果到达 T → 应插入到 T 之前
5. 创建新节点，插入到 $current$ 之前：
 - 若 $prev$ 为空 → 插入头部，修改 $table[b]$
 - 否则 → 插入中间/末尾，修改 $prev.next$
6. 返回插入结果 $inserted, bucket, position$

2.3.3 查找操作

1. 计算桶号 $b = key \% 5$
2. 从 $table[b]$ 开始遍历，同时计数位置
3. 遍历时判断：
 - 如果 $current.key == \text{目标 key}$ → 返回 $found$
 - 如果到达 T → 返回 not_found
4. 返回查找结果 $found/not_found, bucket, position$

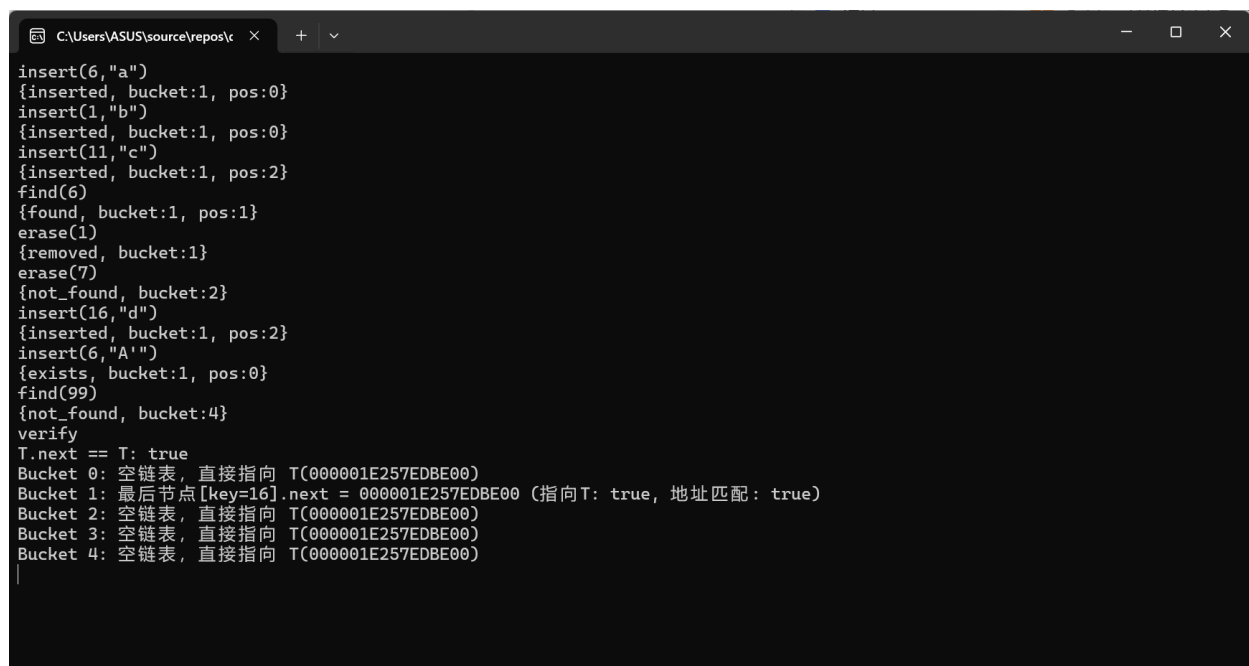
2.3.4 删除操作

1. 计算桶号 $b = \text{key} \% 5$
2. 从 `table[b]` 开始遍历，维护 `prev` 和 `current` 指针
3. 找到目标节点时：
 - 若 `prev` 为空 $\rightarrow$ 删除头节点，修改 `table[b]`
 - 否则 $\rightarrow$ 删除中间节点，修改 `prev.next`
4. 释放节点内存，元素计数减 1
5. 返回删除结果 `removed/not_found`, `bucket`

2.3.5 尾哨兵验证

1. 遍历到每条链的最后一个数据节点
2. 检查该节点的 `next` 指针是否等于 `T` 的地址
3. 输出地址信息供人工验证

2.4 测试用例截图



```
C:\Users\ASUS\source\repos\c > insert(6,"a")
{inserted, bucket:1, pos:0}
C:\Users\ASUS\source\repos\c > insert(1,"b")
{inserted, bucket:1, pos:0}
C:\Users\ASUS\source\repos\c > insert(11,"c")
{inserted, bucket:1, pos:2}
C:\Users\ASUS\source\repos\c > find(6)
{found, bucket:1, pos:1}
C:\Users\ASUS\source\repos\c > erase(1)
{removed, bucket:1}
C:\Users\ASUS\source\repos\c > erase(7)
{not_found, bucket:2}
C:\Users\ASUS\source\repos\c > insert(16,"d")
{inserted, bucket:1, pos:2}
C:\Users\ASUS\source\repos\c > insert(6,"A'")
{exists, bucket:1, pos:0}
C:\Users\ASUS\source\repos\c > find(99)
{not_found, bucket:4}
C:\Users\ASUS\source\repos\c > verify
T.next == T: true
Bucket 0: 空链表, 直接指向 T(000001E257EDBE00)
Bucket 1: 最后节点[key=16].next = 000001E257EDBE00 (指向T: true, 地址匹配: true)
Bucket 2: 空链表, 直接指向 T(000001E257EDBE00)
Bucket 3: 空链表, 直接指向 T(000001E257EDBE00)
Bucket 4: 空链表, 直接指向 T(000001E257EDBE00)
```

2.5 代码

```
1 #include <iostream>
2 #include <string>
3 using namespace std;
```

```
4 template<class K, class E>
5 struct ChainNode {
6     pair<const K, E> data;
7     ChainNode<K, E>* next;
8     ChainNode(const pair<const K, E>& p, ChainNode<K, E>* n = nullptr)
9         : data(p), next(n) {}
10 };
11 template<class K, class E>
12 class hashChainsWithTail {
13 private:
14     static const int capacity = 5;
15     ChainNode<K, E>** table;
16     ChainNode<K, E>* tailNode;
17     int dSize;
18     int hash(const K& key) const {
19         return key % capacity;
20     }
21
22 public:
23     hashChainsWithTail() {
24         dSize = 0;
25         tailNode = new ChainNode<K, E>(pair<const K, E>(K(), E()), nullptr);
26         tailNode->next = tailNode;
27         table = new ChainNode<K, E>* [capacity];
28         for (int i = 0; i < capacity; i++) {
29             table[i] = tailNode;
30         }
31     }
32     ~hashChainsWithTail() {
33         for (int i = 0; i < capacity; i++) {
34             ChainNode<K, E>* current = table[i];
35             while (current != tailNode) {
36                 ChainNode<K, E>* temp = current;
37                 current = current->next;
38                 delete temp;
39             }
40         }
41         delete[] table;
42         delete tailNode;
43     }
44     struct FindResult {
45         bool found;
```

```
46     int bucket;
47     int position;
48     E* valuePtr;
49 };
50
51 FindResult find(const K& theKey) const {
52     FindResult result;
53     result.bucket = hash(theKey);
54     result.position = 0;
55     result.found = false;
56     result.valuePtr = nullptr;
57     ChainNode<K, E>* current = table[result.bucket];
58     while (current != tailNode) {
59         if (current->data.first == theKey) {
60             result.found = true;
61             result.valuePtr = (E*)&current->data.second;
62             return result;
63         }
64         current = current->next;
65         result.position++;
66     }
67     return result;
68 }
69 struct InsertResult {
70     bool inserted;
71     int bucket;
72     int position;
73 };
74
75 InsertResult insert(const pair<const K, E>& thePair) {
76     InsertResult result;
77     result.bucket = hash(thePair.first);
78     result.position = 0;
79     result.inserted = false;
80
81     ChainNode<K, E>* current = table[result.bucket];
82     ChainNode<K, E>* prev = nullptr;
83
84     while (current != tailNode) {
85
86         if (current->data.first == thePair.first) {
87             result.inserted = false;
```



```
88         return result;
89     }
90     if (current->data.first > thePair.first) {
91         break;
92     }
93     prev = current;
94     current = current->next;
95     result.position++;
96 }
97 ChainNode<K, E>* newNode = new ChainNode<K, E>(thePair, current);
98 if (prev == nullptr) {
99     table[result.bucket] = newNode;
100 }
101 else {
102     prev->next = newNode;
103 }
104 dSize++;
105 result.inserted = true;
106 return result;
107 }
108 struct EraseResult {
109     bool removed;
110     int bucket;
111 };
112 EraseResult erase(const K& theKey) {
113     EraseResult result;
114     result.bucket = hash(theKey);
115     result.removed = false;
116     ChainNode<K, E>* current = table[result.bucket];
117     ChainNode<K, E>* prev = nullptr;
118     while (current != tailNode) {
119         if (current->data.first == theKey) {
120             if (prev == nullptr) {
121                 table[result.bucket] = current->next;
122             }
123             else {
124                 prev->next = current->next;
125             }
126             delete current;
127             dSize--;
128             result.removed = true;
129             return result;
```

```
130     }
131     prev = current;
132     current = current->next;
133 }
134 return result;
135 }
136 // 验证尾哨兵 T
137 void verifyTailSentinel() const {
138     cout << "T.next == T: " << (tailNode->next == tailNode ? "true" : "false")
139         << endl;
140
141     for (int i = 0; i < capacity; i++) {
142         ChainNode<K, E>* current = table[i];
143
144         if (current == tailNode) {
145             cout << "Bucket " << i << ": 空链表, 直接指向 T(" << tailNode << ")"
146                 << endl;
147         }
148         else {
149             while (current->next != tailNode) {
150                 current = current->next;
151             }
152             cout << "Bucket " << i << ": 最后节点[key=" << current->data.first
153                 << "].next = " << current->next
154                 << " (指向T: " << (current->next == tailNode ? "true" : "false")
155                 << ", 地址匹配: " << (current->next == tailNode ? "true" :
156                     "false") << ")" << endl;
157         }
158     }
159 }
160
161 int size() const {
162     return dSize;
163 }
164
165 ChainNode<K, E>* getTailNode() const {
166     return tailNode;
167 }
168 };
169
170 void parseCommand(const string& cmd, hashChainsWithTail<int, string>& ht) {
171     string cleanCmd;
172     for (char c : cmd) {
173         if (c != ' ') cleanCmd += c;
174     }
175 }
```

```
169     if (cleanCmd.empty()) return;
170     if (cleanCmd.find("insert(") == 0) {
171         size_t start = cleanCmd.find('(') + 1;
172         size_t comma = cleanCmd.find(',');
173         size_t end = cleanCmd.find(')');
174         if (comma == string::npos || end == string::npos) {
175             cout << "格式错误" << endl;
176             return;
177         }
178         string keyStr = cleanCmd.substr(start, comma - start);
179         string valueStr = cleanCmd.substr(comma + 1, end - comma - 1);
180         if (valueStr.front() == '"' && valueStr.back() == '"') {
181             valueStr = valueStr.substr(1, valueStr.length() - 2);
182         }
183         int key = stoi(keyStr);
184         auto result = ht.insert(make_pair(key, valueStr));
185         if (result.inserted) {
186             cout << "{inserted, bucket:" << result.bucket
187                 << ", pos:" << result.position << "}" << endl;
188         }
189         else {
190             cout << "{exists, bucket:" << result.bucket
191                 << ", pos:" << result.position << "}" << endl;
192         }
193     }
194     else if (cleanCmd.find("find(") == 0) {
195         size_t start = cleanCmd.find('(') + 1;
196         size_t end = cleanCmd.find(')');
197         if (end == string::npos) {
198             cout << "格式错误" << endl;
199             return;
200         }
201         string keyStr = cleanCmd.substr(start, end - start);
202         int key = stoi(keyStr);
203         auto result = ht.find(key);
204         if (result.found) {
205             cout << "{found, bucket:" << result.bucket
206                 << ", pos:" << result.position << "}" << endl;
207         }
208         else {
209             cout << "{not_found, bucket:" << result.bucket << "}" << endl;
210         }
211     }
```

```
211     }
212     else if (cleanCmd.find("erase") == 0) {
213         size_t start = cleanCmd.find('(') + 1;
214         size_t end = cleanCmd.find(')');
215
216         if (end == string::npos) {
217             cout << "格式错误" << endl;
218             return;
219         }
220         string keyStr = cleanCmd.substr(start, end - start);
221         int key = stoi(keyStr);
222         auto result = ht.erase(key);
223
224         if (result.removed) {
225             cout << "{removed, bucket:" << result.bucket << "}" << endl;
226         }
227         else {
228             cout << "{not_found, bucket:" << result.bucket << "}" << endl;
229         }
230     }
231     else if (cleanCmd == "verify" || cleanCmd == "verify()") {
232         ht.verifyTailSentinel();
233     }
234     else {
235         cout << "未知命令 " << cmd << endl;
236     }
237 }
238 int main() {
239     hashChainsWithTail<int, string> ht;
240     string line;
241     while (getline(cin, line)) {
242         if (line == "quit" || line == "exit") {
243             cout << "程序退出" << endl;
244             break;
245         }
246
247         parseCommand(line, ht);
248     }
249     system("pause");
250     return 0;
251 }
```

2.6 时间性能比较

2.6.1 hashChainsWithTail:

1. 尾哨兵模式：每个链表都有一个尾哨兵节点 `tailNode`，链表的最后节点总是指向这个哨兵节点。这个设计的好处在于可以简化一些操作，特别是在链表为空时，避免了额外的 `NULL` 检查。
2. 插入、删除：插入和删除操作通过遍历链表，判断条件是当前节点的 `next` 指向尾节点（哨兵），操作完成后可能会修改前一个节点的 `next` 指向。
3. 查找：链表采用线性查找的方式，遍历该哈希桶中的链表，直到找到匹配的元素或到达尾节点。

2.6.2 hashChains:

1. 固定大小哈希表：使用 `hash(theKey) & divisor` 来定位桶，并且每个桶是一个单独的链表结构。
2. 插入、删除：插入和删除操作是通过链表来实现，但不涉及尾哨兵节点的特殊处理，仅使用链表头进行插入和删除。
3. 查找：查找操作和 `hashChainsWithTail` 类似，也是通过线性遍历桶中的链表来查找目标元素。

操作	hashChainsWithTail	hashChains
最好情况	$O(1)$	$O(1)$
平均情况	$O(n/m)$	$O(n/m)$
最坏情况	$O(n)$	$O(n)$

表 3: 查找操作时间复杂度对比

操作	hashChainsWithTail	hashChains
查找位置	$O(n/m)$	$O(n/m)$
重复检测	在遍历中完成	由容器的 <code>size()</code> 变化判断
插入节点	$O(1)$	$O(1)$
总时间	$O(n/m)$	$O(n/m)$

表 4: 插入操作时间复杂度对比

操作	hashChainsWithTail	hashChains
时间复杂度	$O(n/m)$	$O(n/m)$

表 5: 删除操作时间复杂度对比

n 是元素总数， m 是桶数。